

```
In [1]: import pandas as pd
        from IPython.display import display

        df = pd.read_csv('data.csv')
        num_cols = df.select_dtypes(include='number').columns
        cat_cols = df.select_dtypes(exclude='number').columns
```

```
In [2]: print('Rozmiar zbioru:', df.shape)
        print("WNIOSKI: 418 wierszy to niewiele")
```

Rozmiar zbioru: (418, 20)
WNIOSKI: 418 wierszy to niewiele

```
In [3]: print('\nTypy danych:')
        display(df.dtypes.to_frame('dtype'))
```

Typy danych:

	dtype
ID	int64
N_Days	int64
Status	str
Drug	str
Age	int64
Sex	str
Ascites	str
Hepatomegaly	str
Spiders	str
Edema	str
Bilirubin	float64
Cholesterol	float64
Albumin	float64
Copper	float64
Alk_Phos	float64
SGOT	float64
Tryglicerides	float64
Platelets	float64
Prothrombin	float64
Stage	float64

```
In [4]: missing_info = df.isna().sum().to_frame('missing_values')
        missing_info['percentage'] = (df.isna().mean() * 100).round(2).astype(str) + '%'
```

```
print('\nBraki danych w kolumnach:')
display(missing_info)

print("WNIOSKI: 9 kolumn ma braki danych powyżej 25%")
```

Braki danych w kolumnach:

	missing_values	percentage
ID	0	0.0%
N_Days	0	0.0%
Status	0	0.0%
Drug	106	25.36%
Age	0	0.0%
Sex	0	0.0%
Ascites	106	25.36%
Hepatomegaly	106	25.36%
Spiders	106	25.36%
Edema	0	0.0%
Bilirubin	0	0.0%
Cholesterol	134	32.06%
Albumin	0	0.0%
Copper	108	25.84%
Alk_Phos	106	25.36%
SGOT	106	25.36%
Tryglicerides	136	32.54%
Platelets	11	2.63%
Prothrombin	2	0.48%
Stage	6	1.44%

WNIOSKI: 9 kolumn ma braki danych powyżej 25%

```
In [5]: print('\nPodstawowe statystyki dla cech liczbowych:')
display(df[num_cols].describe().T)

print("WNIOSKI:")
print("najmłodszy pacjent miał 26 lat, natomiast większość pacjentów (25%-75%) j
print("czas obserwacji (n-days) jest bardzo rozpięty, sugeruje to duże zróżnicow
```

Podstawowe statystyki dla cech liczbowych:

	count	mean	std	min	25%	50%	75%
ID	418.0	209.500000	120.810458	1.00	105.2500	209.50	313.75
N_Days	418.0	1917.782297	1104.672992	41.00	1092.7500	1730.00	2613.50
Age	418.0	18533.351675	3815.845055	9598.00	15644.5000	18628.00	21272.50
Bilirubin	418.0	3.220813	4.407506	0.30	0.8000	1.40	3.40
Cholesterol	284.0	369.510563	231.944545	120.00	249.5000	309.50	400.00
Albumin	418.0	3.497440	0.424972	1.96	3.2425	3.53	3.77
Copper	310.0	97.648387	85.613920	4.00	41.2500	73.00	123.00
Alk_Phos	312.0	1982.655769	2140.388824	289.00	871.5000	1259.00	1980.00
SGOT	312.0	122.556346	56.699525	26.35	80.6000	114.70	151.90
Tryglicerides	282.0	124.702128	65.148639	33.00	84.2500	108.00	151.00
Platelets	407.0	257.024570	98.325585	62.00	188.5000	251.00	318.00
Prothrombin	416.0	10.731731	1.022000	9.00	10.0000	10.60	11.10
Stage	412.0	3.024272	0.882042	1.00	2.0000	3.00	4.00

WNIOSKI:

najmłodszy pacjent miał 26 lat, natomiast większość pacjentów (25%-75%) jest w wieku pomiędzy 42-58 lat

czas obserwacji (n-days) jest bardzo rozpięty, sugeruje to duże zróżnicowanie postępowania choroby/bardzo różne fazy rozpoznania choroby i udania się do szpitala

```
In [6]: print(df['Stage'].value_counts().sort_index())
print("Widzimy że zdecydowana większość pacjentów w zbiorze jest w ciężkim stani
```

Stage

1.0 21

2.0 92

3.0 155

4.0 144

Name: count, dtype: int64

Widzimy że zdecydowana większość pacjentów w zbiorze jest w ciężkim stanie (3 i 4 stadium choroby)

```
In [7]: for col in cat_cols:
print(f"\nCecha: {col}")
counts = df[col].value_counts(dropna=False)
percents = df[col].value_counts(normalize=True, dropna=False) * 100
summary = pd.concat([counts, percents.round(2)], axis=1, keys=['Liczba', 'Pr
display(summary)

print("WNIOSKI")
print("zdecydowana większość wierszy to kobiety")
print("liczność ludzi przyjmujących placebo i d-pecicylamine jest prawie równe")
print("bardzo mało osób miało przeszczep, prawie 40% przypadków ze zbioru zakońc
```

Cecha: Status

	Liczba	Procent %
Status		
C	232	55.50
D	161	38.52
CL	25	5.98

Cecha: Drug

	Liczba	Procent %
Drug		
D-penicillamine	158	37.80
Placebo	154	36.84
NaN	106	25.36

Cecha: Sex

	Liczba	Procent %
Sex		
F	374	89.47
M	44	10.53

Cecha: Ascites

	Liczba	Procent %
Ascites		
N	288	68.90
NaN	106	25.36
Y	24	5.74

Cecha: Hepatomegaly

	Liczba	Procent %
Hepatomegaly		
Y	160	38.28
N	152	36.36
NaN	106	25.36

Cecha: Spiders

	Liczba	Procent %
Spiders		
N	222	53.11
NaN	106	25.36
Y	90	21.53

Cecha: Edema

	Liczba	Procent %
Edema		
N	354	84.69
S	44	10.53
Y	20	4.78

WNIOSKI

zdecydowana większość wierszy to kobiety

liczność ludzi przyjmujących placebo i d-pecicylamine jest prawie równe

bardzo mało osób miało przeszczep, prawie 40% przypadków ze zbioru zakończyło się śmiercią

```
In [8]: print('\nRozkład etykiety Stage:')
display(df['Stage'].value_counts(dropna=False).sort_index().to_frame('count'))
print("widzimy że zdecydowana większość przypadków w zbiorze to przypadki o dwóch
```

Rozkład etykiety Stage:

	count
Stage	
1.0	21
2.0	92
3.0	155
4.0	144
NaN	6

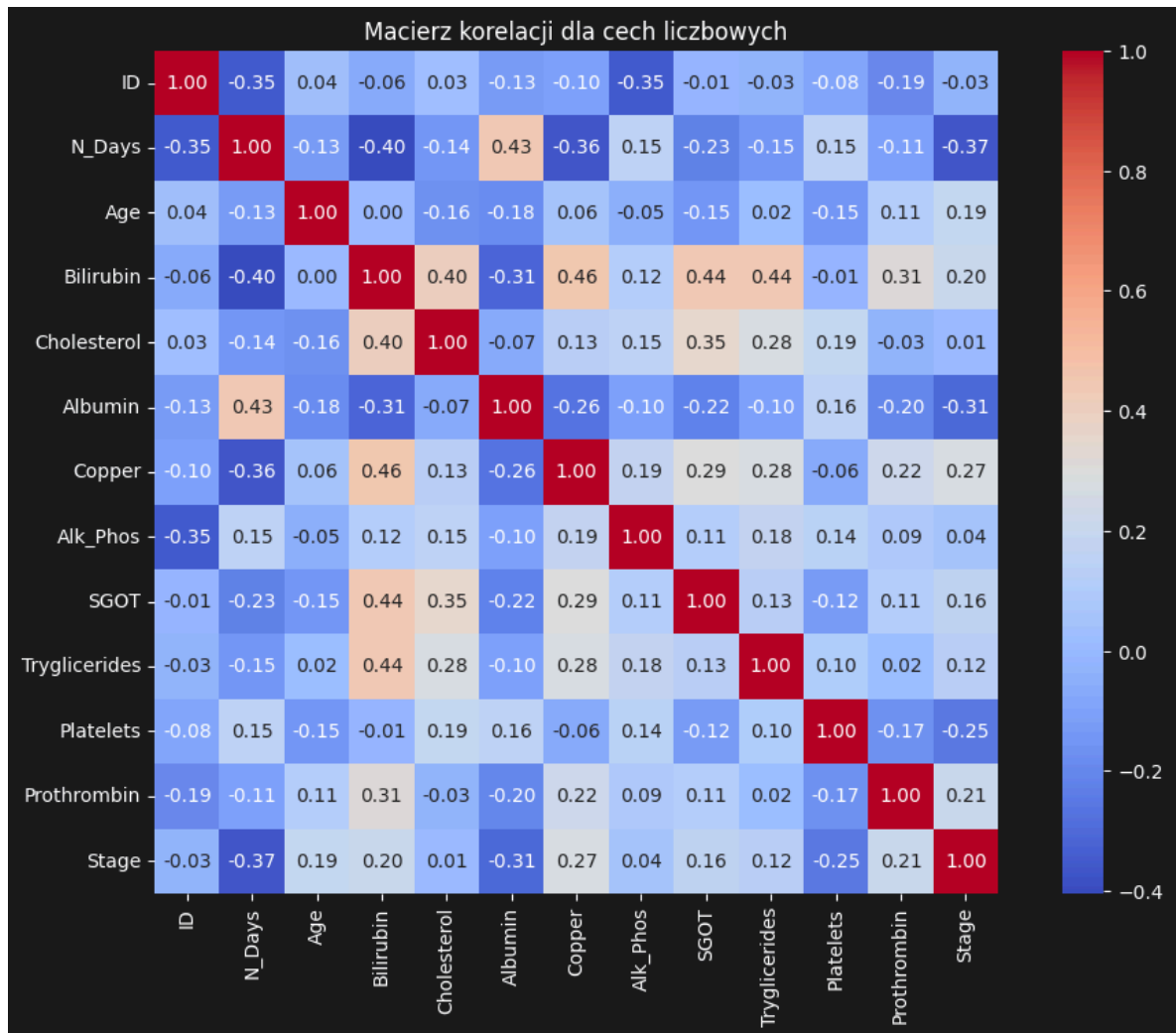
widzimy że zdecydowana większość przypadków w zbiorze to przypadki o dwóch najwyższych stadiach choroby, co może tłumaczyć wysoką śmiertelność, która była wnioskiem w poprzedniej komórce

```
In [9]: import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(12, 8))
corr_matrix = df[num_cols].corr()
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt=".2f", square=True)
plt.title('Macierz korelacji dla cech liczbowych')
plt.show()

print("WNIOSKI:")
print("1. Występuje zauważalna dodatnia korelacja (0.43) między poziomem albumin
```

```
print("2. Czas (N_Days) jest ujemnie skorelowany z poziomem bilirubiny (-0.40),  
print("3. Zmienne takie jak Wiek (Age) oraz Płytki krwi (Platelets) są bardzo sł
```



WNIOSKI:

1. Występuje zauważalna dodatnia korelacja (0.43) między poziomem albumin a liczbą dni (N_Days). Może to oznaczać, że pacjenci z wyższym poziomem tego białka charakteryzują się dłuższym okresem przeżycia (lub dłuższą obserwacją w badaniu).
2. Czas (N_Days) jest ujemnie skorelowany z poziomem bilirubiny (-0.40), stadiem choroby (Stage: -0.37) oraz poziomem miedzi (-0.36). Im bardziej zaawansowane stadium i wyższe stężenie tych toksycznych substancji, tym krótszy czas obserwacji pacjenta, co najprawdopodobniej wiąże się z szybszym zgonem.
3. Zmienne takie jak Wiek (Age) oraz Płytki krwi (Platelets) są bardzo słabo skorelowane z większością pozostałych cech. Oznacza to, że wiek pacjenta nie determinuje wprost wartości tych wyników biochemicznych w tym konkretnym zbiorze danych.

```
In [10]: crosstab_drug = pd.crosstab(df['Drug'], df['Status'], normalize='index') * 100  
display(crosstab_drug.round(2).astype(str) + '%')  
  
drug_stats = df.groupby('Drug')[['Bilirubin', 'Prothrombin', 'Copper']].mean()  
display(drug_stats.round(2))  
  
print("WNIOSKI:")  
print("1. Branie leku vs placebo nie wpływa znacząco na przeżywalność pacjenta")  
print("2. Średni poziom bilirubiny jest niższy u pacjentów leczonych D-penicylam
```

Status	C	CL	D
Drug			
D-penicillamine	52.53%	6.33%	41.14%
Placebo	55.19%	5.84%	38.96%

	Bilirubin	Prothrombin	Copper
Drug			
D-penicillamine	2.87	10.65	97.64
Placebo	3.65	10.80	97.65

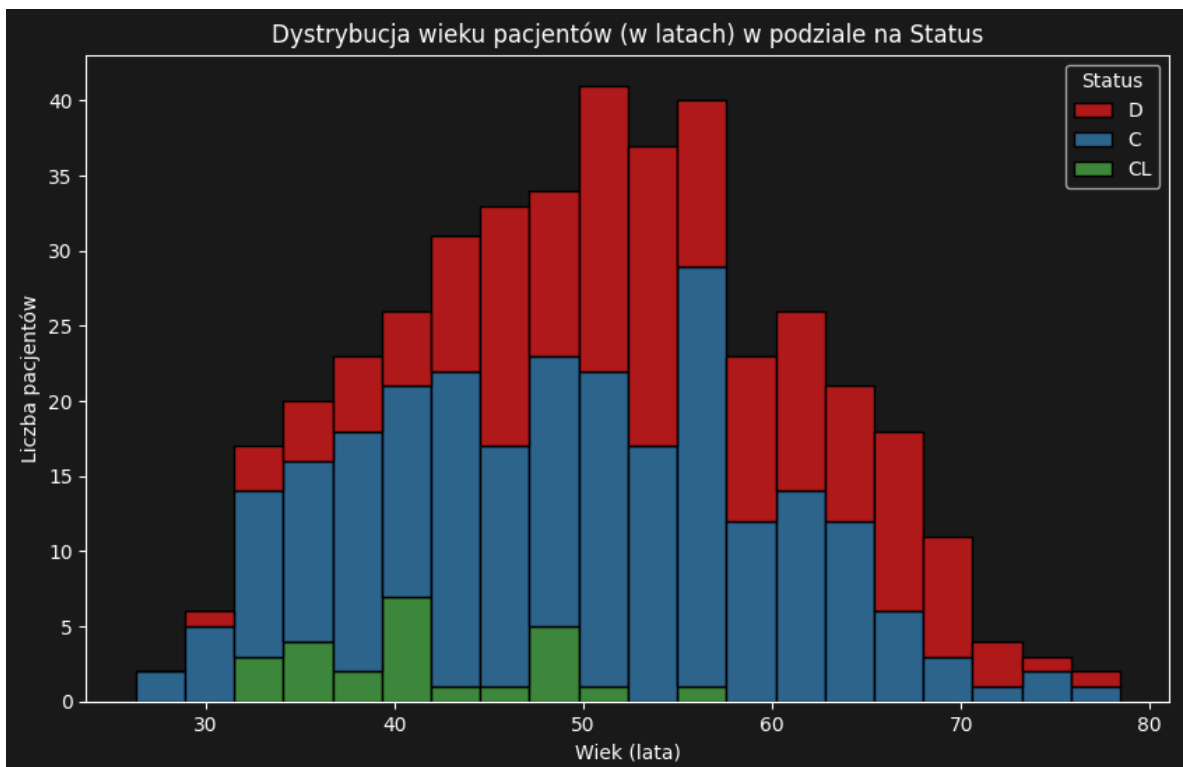
WNIOSKI:

1. Branie leku vs placebo nie wpływa znacząco na przeżywalność pacjenta
2. Średni poziom bilirubiny jest niższy u pacjentów leczonych D-penicylaminą (2.87) w porównaniu do grupy placebo (3.65), co może sugerować, że lek ma łagodzący wpływ na ten konkretny parametr wątrobowy.

```
In [11]: df['Age_years'] = df['Age'] / 365.25

plt.figure(figsize=(10, 6))
sns.histplot(data=df, x='Age_years', hue='Status', multiple='stack', bins=20, pa
plt.title('Dystrybucja wieku pacjentów (w latach) w podziale na Status')
plt.xlabel('Wiek (lata)')
plt.ylabel('Liczba pacjentów')
plt.show()

print("WNIOSKI:")
print("- Rozkład zgonów (Status D) jest zauważalnie przesunięty w prawo, co ozna
print("- W grupie pacjentów nie seniorów (przed 60), znacznie częściej obserwuje
```



WNIOSKI:

- Rozkład zgonów (Status D) jest zauważalnie przesunięty w prawo, co oznacza, że starsi pacjenci (szczególnie powyżej 45 roku życia) stanowią największą grupę ryzyka.
- W grupie pacjentów nie seniorów (przed 60), znacznie częściej obserwujemy status C (pacjenci ocalejący/żyjący na koniec badań).

```

In [ ]: import pandas as pd
        from sklearn.model_selection import train_test_split
        from sklearn.impute import KNNImputer
        from sklearn.preprocessing import MinMaxScaler

df = pd.read_csv('data.csv')

cat_cols = df.select_dtypes(include=['object', 'category', 'string']).columns
for col in cat_cols:
    df[col] = df[col].fillna('Unknown')

df_train, df_val = train_test_split(df, test_size=0.2, random_state=42)

num_cols = df.select_dtypes(include=['number']).columns.drop('ID')

scaler = MinMaxScaler()
X_train_scaled = scaler.fit_transform(df_train[num_cols])
X_val_scaled = scaler.transform(df_val[num_cols])

imputer = KNNImputer(n_neighbors=5)
X_train_imputed_scaled = imputer.fit_transform(X_train_scaled)
X_val_imputed_scaled = imputer.transform(X_val_scaled)

X_train_imputed = scaler.inverse_transform(X_train_imputed_scaled)
X_val_imputed = scaler.inverse_transform(X_val_imputed_scaled)

df_train_imputed = pd.DataFrame(X_train_imputed, columns=num_cols, index=df_train.index)
df_val_imputed = pd.DataFrame(X_val_imputed, columns=num_cols, index=df_val.index)

if 'Stage' in num_cols:
    df_train_imputed['Stage'] = df_train_imputed['Stage'].round()
    df_val_imputed['Stage'] = df_val_imputed['Stage'].round()

df_train_final = df_train.copy()
df_val_final = df_val.copy()

for col in num_cols:
    df_train_final[col] = df_train_final[col].fillna(df_train_imputed[col])
    df_val_final[col] = df_val_final[col].fillna(df_val_imputed[col])

df_train_final.to_csv('pbc_train_imputed.csv', index=False)
df_val_final.to_csv('pbc_val_imputed.csv', index=False)

```

W kolumnach kategoriycznych dodaje wartość "unknown" jako osobną klasę wartości.
Natomiast dla cech numerycznych używam knn

```

In [ ]: import numpy as np
        import pandas as pd
        from sklearn.preprocessing import StandardScaler, MinMaxScaler, KBinsDiscretizer
        from sklearn.feature_selection import SelectKBest, f_classif
        from sklearn.decomposition import PCA
        from sklearn.tree import DecisionTreeClassifier
        from sklearn.linear_model import LogisticRegression
        from sklearn.metrics import accuracy_score

X_train_raw_num = df_train_final[num_cols].values
X_val_raw_num = df_val_final[num_cols].values

```

```

cat_features = [c for c in cat_cols if c != 'Status']
encoder = OneHotEncoder(handle_unknown='ignore', sparse_output=False)
X_train_cat = encoder.fit_transform(df_train_final[cat_features])
X_val_cat = encoder.transform(df_val_final[cat_features])

y_train = df_train_final['Status'].values
y_val = df_val_final['Status'].values

X_train_baseline = np.hstack([X_train_raw_num, X_train_cat])
X_val_baseline = np.hstack([X_val_raw_num, X_val_cat])

results = {}

def evaluate(X_tr, X_va, name):
    lr = LogisticRegression(solver='newton-cg', random_state=42, max_iter=1000)
    lr.fit(X_tr, y_train)
    acc_lr = accuracy_score(y_val, lr.predict(X_va))
    dt = DecisionTreeClassifier(random_state=42)
    dt.fit(X_tr, y_train)
    acc_dt = accuracy_score(y_val, dt.predict(X_va))
    results[name] = {'Regresja Logistyczna': acc_lr, 'Drzewo Decyzyjne': acc_dt}

evaluate(X_train_baseline, X_val_baseline, 'Bez przetwarzania (Baseline)')

norm = MinMaxScaler()
X_tr_norm = np.hstack([norm.fit_transform(X_train_raw_num), X_train_cat])
X_va_norm = np.hstack([norm.transform(X_val_raw_num), X_val_cat])
evaluate(X_tr_norm, X_va_norm, 'Normalizacja')

std = StandardScaler()
X_tr_std_num = std.fit_transform(X_train_raw_num)
X_va_std_num = std.transform(X_val_raw_num)
X_tr_std = np.hstack([X_tr_std_num, X_train_cat])
X_va_std = np.hstack([X_va_std_num, X_val_cat])
evaluate(X_tr_std, X_va_std, 'Standaryzacja')

disc = KBinsDiscretizer(n_bins=5, encode='ordinal', strategy='uniform', subsample=
X_tr_disc = np.hstack([disc.fit_transform(X_train_raw_num), X_train_cat])
X_va_disc = np.hstack([disc.transform(X_val_raw_num), X_val_cat])
evaluate(X_tr_disc, X_va_disc, 'Dyskretyzacja')

sel = SelectKBest(score_func=f_classif, k=10)
X_tr_sel = sel.fit_transform(X_train_baseline, y_train)
X_va_sel = sel.transform(X_val_baseline)
evaluate(X_tr_sel, X_va_sel, 'Selekcja cech')

pca = PCA(n_components=5, random_state=42)
X_tr_pca = np.hstack([pca.fit_transform(X_tr_std_num), X_train_cat])
X_va_pca = np.hstack([pca.transform(X_va_std_num), X_val_cat])
evaluate(X_tr_pca, X_va_pca, 'PCA')

res_df = pd.DataFrame(results).T
res_df

```

```

In [ ]: from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier

```

```

results_advanced = []

def evaluate_all_metrics(X_tr, X_va, name):
    lr = LogisticRegression(solver='newton-cg', random_state=42, max_iter=1000)
    lr.fit(X_tr, y_train)
    y_pred_lr = lr.predict(X_va)

    dt = DecisionTreeClassifier(random_state=42)
    dt.fit(X_tr, y_train)
    y_pred_dt = dt.predict(X_va)

    models = [('Regresja Logistyczna', y_pred_lr), ('Drzewo Decyzyjne', y_pred_d

    for model_name, y_pred in models:
        results_advanced.append({
            'Przetwarzanie danych': name,
            'Model': model_name,
            'ACC': accuracy_score(y_val, y_pred),
            'TPR': recall_score(y_val, y_pred, average='weighted', zero_division
            'PPV': precision_score(y_val, y_pred, average='weighted', zero_divis
            'F1-score': f1_score(y_val, y_pred, average='weighted', zero_divisio
        })

evaluate_all_metrics(X_train_baseline, X_val_baseline, 'Baseline')
evaluate_all_metrics(X_tr_norm, X_va_norm, 'Normalizacja')
evaluate_all_metrics(X_tr_std, X_va_std, 'Standaryzacja')
evaluate_all_metrics(X_tr_disc, X_va_disc, 'Dyskretyzacja')
evaluate_all_metrics(X_tr_sel, X_va_sel, 'Selekcja cech')
evaluate_all_metrics(X_tr_pca, X_va_pca, 'PCA')

res_advanced_df = pd.DataFrame(results_advanced)
res_advanced_df

```

Baseline - surowe wartości liczbowe plus kategoryczne jako onehot encoding

normalizacja (minmaxscaler) - skaluje numeryczne do (0,1)

standaryzacja (standard scaler) - zmienia tak aby srednia wyniosla 0, a odchylenie 1. Ma to pomóc z outliers

dyskretyzacja (KBinsDiscretizer) - zmienia wartości ciągłe na koszyki

Selekcja cech (SelectKBest) - zamiast modyfikować wartości, algorytm wybiera 10 najbardziej wartościowych cech na podstawie testu statystycznego

PCA (Analiza Głównych Składowych) - metoda redukcji wymiarowości. Tworzy nowe, sztuczne cechy (tutaj 5), które są liniowymi kombinacjami oryginalnych cech numerycznych. Celem jest zachowanie jak największej ilości informacji (wariancji) przy jednoczesnym zmniejszeniu liczby kolumn.

DRZEWO DECYZYJNE Wyniki eksperymentu wykazują brak wpływu skalowania danych, ponieważ zarówno normalizacja, jak i standaryzacja dały dokładnie ten sam rezultat co model bazowy, osiągając wartość 0.607. Jest to jednak całkowicie prawidłowe i oczekiwane zachowanie dla zastosowanego algorytmu. Drzewa decyzyjne są w pełni

niewrażliwe na monotoniczne transformacje, co oznacza, że punkt podziału po prostu przesuwają się razem ze skalą, podczas gdy sama struktura drzewa pozostaje identyczna.

Największym sukcesem okazało się zastosowanie dyskretyzacji oraz selekcji cech, które przyniosły odpowiednio wyniki na poziomie 0.750 i 0.738, będąc najlepszymi rezultatami dla tego modelu. Dyskretyzacja drastycznie ograniczyła liczbę możliwych punktów podziału, wymuszając na drzewie zdolność do większego uogólniania. Dodatkowo selekcja cech skutecznie usunęła szum, przez co algorytm stracił możliwość przeuczenia się na zmiennych o znikomym znaczeniu.

Z kolei analiza głównych składowych (PCA) przyniosła jedynie średni wynik na poziomie 0.654. Choć algorytm ten skompresował dane i zmniejszył zjawisko przeuczenia, dając rezultat lepszy niż model bazowy, to obnażył pewne ograniczenia zastosowanego klasyfikatora. Drzewa decyzyjne z reguły słabo radzą sobie ze skośnymi podziałami wynikającymi z kombinacji liniowych tworzonych przez PCA, zdecydowanie preferując pracę na oryginalnych cechach, które mogą być dzielone w sposób prostopadły.

REGRESJA LOGISTYCZNA Najlepszym podejściem okazało się zastosowanie normalizacji, która przyniosła najwyższy wynik na poziomie 0.833. Modele liniowe z regularyzacją wykazują ogromną wrażliwość na skalę danych, przez co przy zróżnicowanych rzędach wielkości cech algorytm faworyzuje te o mniejszych wartościach, przypisując im sztucznie większe wagi. Sprowadzenie wszystkich zmiennych do sztywnego przedziału od 0 do 1 idealnie zgrało się z mechanizmem regularyzacji dla tego konkretnego zbioru, co w konsekwencji pozwoliło solverowi na optymalne i sprawiedliwe dopasowanie wag modelu.

Zupełnie inaczej sytuacja wyglądała w przypadku standaryzacji, która dała wynik 0.785, czyli dokładnie taki sam jak w wariancie bazowym. Oznacza to, że wyśrodkowanie danych wokół zera przy zachowaniu wariancji równej 1 nie przyniosło solverowi większych korzyści niż praca na surowych informacjach. Główną przyczyną takiego stanu rzeczy jest prawdopodobnie obecność wartości odstających w zbiorze danych, które po standaryzacji wciąż negatywnie wpływały na wagi modelu, podczas gdy wspomniana wcześniej normalizacja zamknęła je sztywno w granicy jedności.

Zadowolające rezultaty przyniosły również zabiegi dyskretyzacji oraz selekcji cech, osiągając wynik na poziomie 0.797. Regresja logistyczna z natury zakłada liniową relację między cechą a wynikiem, jednak zastosowanie dyskretyzacji pozwoliło modelowi uchwycić zależności nieliniowe, w których skrajne wartości cechy mogą mieć inny wpływ na wynik niż wartości średnie. Dodatkowo proces selekcji skutecznie usunął szum informacyjny oraz cechy silnie skorelowane, co przełożyło się na lekką poprawę ogólnej skuteczności predykcji.

```
In [ ]: from sklearn.naive_bayes import GaussianNB
        from sklearn.tree import DecisionTreeClassifier
        from sklearn.model_selection import GridSearchCV
        from sklearn.metrics import accuracy_score
        import pandas as pd

        nb_classifier = GaussianNB()
```

```

nb_param_grid = {
    'var_smoothing': [1e-9, 1e-7, 1e-5, 1e-3, 1e-1]
}

nb_grid_search = GridSearchCV(estimator=nb_classifier, param_grid=nb_param_grid,
nb_grid_search.fit(X_tr_std, y_train)

dt_classifier = DecisionTreeClassifier(random_state=42)
dt_param_grid = {
    'criterion': ['gini', 'entropy'],
    'max_depth': [None, 3, 5, 10],
    'min_samples_split': [2, 5, 10, 20]
}

dt_grid_search = GridSearchCV(estimator=dt_classifier, param_grid=dt_param_grid,
dt_grid_search.fit(X_tr_std, y_train)

nb_best_model = nb_grid_search.best_estimator_
dt_best_model = dt_grid_search.best_estimator_

nb_val_predictions = nb_best_model.predict(X_va_std)
dt_val_predictions = dt_best_model.predict(X_va_std)

nb_val_accuracy = accuracy_score(y_val, nb_val_predictions)
dt_val_accuracy = accuracy_score(y_val, dt_val_predictions)

tuning_results = pd.DataFrame({
    'Algorytm': ['Naiwny klasyfikator Bayesa', 'Drzewo decyzyjne'],
    'Najlepsze hiperparametry': [str(nb_grid_search.best_params_), str(dt_grid_s
    'Dokładność (Zbiór walidacyjny)': [nb_val_accuracy, dt_val_accuracy]
})
pd.set_option('display.max_colwidth', None)
tuning_results

```

```

In [ ]: from sklearn.metrics import precision_score, recall_score, f1_score
import pandas as pd

metrics_data = []

models_preds = [
    ('Naiwny klasyfikator Bayesa', nb_val_predictions),
    ('Drzewo decyzyjne', dt_val_predictions)
]

for model_name, y_pred in models_preds:
    acc = accuracy_score(y_val, y_pred)
    prec = precision_score(y_val, y_pred, average='weighted', zero_division=0)
    rec = recall_score(y_val, y_pred, average='weighted', zero_division=0)
    f1 = f1_score(y_val, y_pred, average='weighted', zero_division=0)

    metrics_data.append({
        'Model': model_name,
        'Dokładność (ACC)': acc,
        'Czułość (TPR)': rec,
        'Precyzja (PPV)': prec,
        'F1-score': f1
    })

```

```
evaluation_df = pd.DataFrame(metrics_data)
evaluation_df
```

Parametry:

1. `var_smoothing` odpowiada za sztuczne rozszerzenie (wygładzenie) wariancji cech.
2. `criterion` definiuje funkcję mierzącą jakość podziału w każdym węźle drzewa. gini jest szybszy obliczeniowo, podczas gdy entropia jest nieco bardziej precyzyjny
3. `maxdepth` - max głębokość drzewa
4. `min_samples_split` to minimalna liczba próbek (wierszy danych), jaka musi znajdować się w danym węźle, aby algorytm w ogóle spróbował podzielić go na kolejne gałęzie

Dla naiwnego klasyfikatora Bayesa najlepsze wyniki uzyskano dla parametru wygładzania wariancji (`var_smoothing`) równego 0.1. Taka wartość, będąca najwyższą z testowanej puli, sugeruje, że dodanie większej wartości do wariancji rozkładu danych treningowych pomogło w stabilizacji modelu i redukcji szumu. Dzięki temu zabiegowi klasyfikator osiągnął najwyższą dokładność na zbiorze walidacyjnym, wynoszącą około 72,6%, co czyni go nieznacznie skuteczniejszym w analizowanym problemie.

Dla algorytmu drzewa decyzyjnego optymalna okazała się konfiguracja wykorzystująca entropię jako kryterium podziału węzłów, maksymalną głębokość drzewa ograniczoną do 5 oraz minimalną liczbę próbek do podziału równą 2. Ograniczenie głębokości do pięciu poziomów miało kluczowe znaczenie, ponieważ zapobiegło to nadmiernemu dopasowaniu modelu do danych treningowych (przeuczeniu), pozwalając jednocześnie na uchwycenie najważniejszych zależności.

Użyte biblioteki: pandas - biblioteka do manipulacji, przetwarzania i analizy tabelarycznych danych strukturalnych, scikit learn - narzędzie dostarczające gotowe algorytmy uczenia maszynowego oraz funkcje do przygotowywania danych i oceny modeli, matplotlib - tworzenie wykresów, seaborn - nakładka do tworzenia ładniejszych wykresów